RAJARATA UNIVERSITY OF SRI LANKA

Department of Computing • Faculty of Applied Sciences

ICT 1306 — Object Oriented Programming (Group Assignment)

FixSphere: Mobile Phone Repair Center Management System

Desktop Application Built with C++/CLI (Windows Forms) & MySQL Database

Key Submission Links:
• **GitHub Repository URL:** https://github.com/Ravindu2k4/Fixsphere (Publicly Accessible)
• **5-Minute Live Video Demo (YouTube Unlisted):** https://youtu.be/Y7snjMJPYmc?si=pkCCW51Er08VB-T-

1. Team & Scenario Details

1.1 Team Member Details (Team of 5)

#	Student Full Name	Index Number	Registration Number	Assigned Role & Contribution
1	**K. R. M. De Silva**	6675	ICT/2024/063	Project Leader, System Architecture, Database Schema
2	**S. P. Abewickrama**	6618	ICT/2024/003	Windows Forms UI, Navigation, Dashboard Layout
3	**D. L. C. A. Liyanawaduge**	6685	ICT/2024/078	OOP Domain Classes (Person, Customer, Employee)
4	**K. W. R. Adithya**	6620	ICT/2024/006	Role Permissions, Status Updates, CRUD Logic
5	**M. J. Apeksha**	6628	ICT/2024/014	LKR Currency Calculations, Monthly Reports, Testing

1.2 Chosen Business Scenario

We selected a **Mobile Phone Repair and Servicing Center** named **FixSphere**. In this business, customers bring broken mobile phones (broken screens, battery drainage, water damage, port issues). FixSphere manages the complete intake, diagnosis, repair assignment, and billing process.

The Problem with Manual Systems: Small repair shops often use paper notebooks. This leads to lost customer contact numbers, confusion about which technician is repairing which phone, mistakes in billing, and no security permissions between front-desk staff and technicians.

FixSphere Desktop Solution: A multi-role desktop application built in Visual Studio using C++/CLI and a real MySQL database. It secures the business through 3 roles:

- **Admin:** Has full control — can register users, assign jobs to technicians, delete jobs, and view total revenue reports.
- **Checker:** Front-desk staff — registers customers, takes in new phone repair jobs, and searches records. Cannot change repair statuses or assign jobs.
- **Repairer:** Technician — can only see and update jobs assigned to their own account, set diagnoses, and input repair costs in Sri Lankan Rupees (LKR).

2. Class List and Responsibilities

To meet the assignment requirements, our project contains modular classes separating the user interface, the business entities, and database communication.

Class Name	File Location	Responsibilities & Key Methods
Person Abstract Class	`Models.h`	Acts as the base template for any person in the system. • Holds common private data: `id`, `name`, `contact`, `nic`. • Encapsulates them with public properties (get/set). • Declares pure abstract method: `virtual String^ GetRoleDescription() = 0;`. • Provides base summary method: `virtual String^ GetSummary();`.
Customer Derived Class	`Models.h`	Inherits from `Person`. • Represents a client who brings a phone for repair. • Adds customer-specific data: `address`. • Overrides `GetRoleDescription()` to return customer role details. • Overrides `GetSummary()` to include home address.
Employee Derived Class	`Models.h`	Inherits from `Person`. • Represents a staff member working in the shop. • Adds employee-specific data: `role` (Admin/Checker/Repairer), `username`, `isActive`. • Overrides `GetRoleDescription()` to return the staff designation. • Overrides `GetSummary()` to include role and login username.
UserSession	`UserSession.h`	Tracks the currently logged-in user in memory. • Stores current employee ID, name, username, and role. • Provides role-permission helper methods: `CanManageCustomers()`, `CanCreateJobs()`, `CanUpdateJobStatus()`, `CanAssignRepairer()`, `CanDeleteJobs()`. • Provides `Logout()` to securely clear current session.
DatabaseService	`DatabaseService.h`	Manages real MySQL database connectivity and security. • Reads connection settings (host, port, user, password) from `db_config.ini`. • Provides `GetConnection()` to open MySQL connections. • Provides `ExecuteQuery()` returning safe `DataTable^`. • Provides `ExecuteNonQuery()` for INSERT, UPDATE, DELETE operations. • Encrypts passwords using standard SHA-256 algorithm.
ComboItem	`MainForm.h`	Helper item for ComboBox dropdown controls. • Stores an integer database ID (`Id`) and a human-readable

Class Name	File Location	Responsibilities & Key Methods
		name (`Display`). • Overrides `ToString()` for polymorphic dropdown rendering.
LoginForm	`LoginForm.h`	Authentication screen (Screen 1). • Captures username and password. • Hashes input with SHA-256 and verifies against `employees` table. • Sets `UserSession::CurrentUser` on success.
MainForm	`MainForm.h`	Main multi-tab dashboard (Screens 2 to 6). • Houses views: Dashboard, Customers, Repair Jobs, Employees, Reports. • Runs `ApplyRoleSecurity()` to hide/disable unauthorized buttons. • Manages full CRUD operations and LKR financial summaries.

3. Worked Examples of Each OOP Pillar from FixSphere Code

The assignment requires demonstrating all **6 OOP concepts** using real code written in our application: (1) Class, (2) Object, (3) Encapsulation, (4) Abstraction, (5) Inheritance, and (6) Polymorphism.

3.1 Pillar 1: Class

> **Concept:** A class is a blueprint or template that defines variables (attributes) and functions (behaviors).

Code Example from `Models.h`:

```
// Class blueprint for Customer inheriting common person details
public ref class Customer : public Person
{
private:
    String^ address;

public:
    // Constructor initializing customer fields
    Customer(int id, String^ name, String^ contact, String^ nic, String^ address)
        : Person(id, name, contact, nic)
    {
        this->address = address;
    }

    property String^ Address
    {
        String^ get() { return address; }
        void set(String^ value) { address = value; }
    }
};
```

Explanation: The `Customer` class defines what data a customer holds (such as their home address) and what actions they can perform.

3.2 Pillar 2: Object

> **Concept:** An object is an actual instance of a class created in computer memory during program execution using the gcnew keyword.

Code Example from `LoginForm.h` and `MainForm.h`:

```
// Creating a concrete UserSession object when an employee logs in
UserSession^ session = gcnew UserSession(empId, fullName, username, role);
UserSession::CurrentUser = session;

// Creating a concrete Customer object dynamically in memory
Customer^ custObj = gcnew Customer(1, L"Kamal Perera", L"0771234567",
                                   L"200012345678", L"Anuradhapura");
```

Explanation: Here, `session` and `custObj` are real objects created in memory with specific personal values.

3.3 Pillar 3: Encapsulation

> **Concept:** Encapsulation keeps internal data safe by making fields `private` and allowing access only through controlled `public` properties (getters/setters).

Code Example from `Models.h`:

```
public ref class Person abstract
{
private:
    // Private fields: cannot be modified directly from outside
    int id;
    String^ name;
    String^ contact;
    String^ nic;

public:
    // Controlled public property for Contact
    property String^ Contact
    {
        String^ get() { return contact; }
        void set(String^ value)
        {
            if (!String::IsNullOrWhiteSpace(value))
                contact = value;
        }
    }
};
```

Explanation: Other parts of the program cannot accidentally corrupt `contact` or `name` because the variables are private. They can only be accessed through safe properties.

3.4 Pillar 4: Abstraction

> **Concept:** Abstraction hides complex internal implementation details and shows only the essential interface to the user.

Code Example 1 (Abstract Class & Pure Virtual Method in `Models.h`):

```
public ref class Person abstract
{
public:
    // Pure abstract method: enforces every child class to implement its own role logic
    virtual String^ GetRoleDescription() abstract;
};
```

Code Example 2 (Hiding Database Complexity in `DatabaseService.h`):

```
// In the UI form, the developer simply calls this one-line abstraction:
DataTable^ dt = DatabaseService::ExecuteQuery(
    "SELECT * FROM repair_jobs WHERE status = @st", params);
```

Explanation: The UI form does not know how connection strings, MySQL sockets, open/close connections, or command objects work. All of that complex low-level work is completely hidden behind `DatabaseService`.

3.5 Pillar 5: Inheritance

> **Concept:** Inheritance allows a new class (child class) to inherit variables and methods from an existing class (parent class), avoiding duplicate code.

Code Example from `Models.h`:

```
// Employee inherits common personal attributes from Person base class
public ref class Employee : public Person
{
private:
    String^ role;
    String^ username;
    bool isActive;

public:
    // Calls the base Person constructor using : Person(id, name, contact, nic)
    Employee(int id, String^ name, String^ contact, String^ nic,
             String^ role, String^ username, bool isActive)
        : Person(id, name, contact, nic)
    {
        this->role = role;
        this->username = username;
        this->isActive = isActive;
    }
};
```

Explanation: Both `Customer` and `Employee` inherit `id`, `name`, `contact`, and `nic` directly from `Person` without having to write those fields twice.

3.6 Pillar 6: Polymorphism

> **Concept:** Polymorphism means "many forms". It allows derived classes to provide their own version of a virtual method using the `override` keyword, called dynamically at runtime.

Code Example from `Models.h`:

```
// 1. Customer provides its own implementation
virtual String^ Customer::GetRoleDescription() override
{
    return L"Client / Device Owner requiring phone hardware or software servicing.";
}

// 2. Employee provides a completely different implementation of the same method
virtual String^ Employee::GetRoleDescription() override
{
    return String::Format(L"Internal Staff Member operating under role: {0}", role);
}

// 3. Runtime Polymorphic Behavior:
Person^ personRef = gcnew Employee(1, L"Admin", L"0770000000", L"199000000000",
                                   L"Admin", L"admin", true);
String^ desc = personRef->GetRoleDescription(); // Calls Employee's method dynamically
```

Explanation: Even though `personRef` is a base `Person^` pointer, the program calls the specific `Employee` method at runtime.

4. Object-Oriented Class Diagram

The class diagram below represents the complete object-oriented structure of FixSphere, showing inheritance, encapsulation, abstract methods, and relationships.




| - txtPassword: TextBox^
| - btnSignIn: Button^
| + btnSignIn_Click(...)
| - dgvJobs: DataGridView^
| - dgvCustomers: DataGridView^
| + ApplyRoleSecurity()
| + LoadDashboardData()
| + btnJobSave_Click(...)
| + btnExportReport_Click(...)

5. Real MySQL Database Connection & Full CRUD Implementation

The assignment mandates connecting to a real database (MySQL) and demonstrating full **CRUD (Create, Read, Update, Delete)** operations across at least two record types. FixSphere manages three distinct tables:

5.1 Relational Database Schema

```
[customers] 1 <------------------>> ∞ [repair_jobs] ∞ <<------------------> 1 [employees]
(customer_id PK)                  (job_id PK)                          (emp_id PK)
```

5.2 Demonstration of Full CRUD Operations

Record Type	CREATE (Add)	READ (View & Search)	UPDATE (Edit)	DELETE (Remove)
1. Customers	Adds new customer with Name, Contact (10 digits), unique NIC, and Address.	Real-time search by Customer Name, Phone, or NIC in `DataGridView`.	Updates customer address and contact number directly from form.	Admin can safely delete customer records with relational integrity checks.
2. Repair Jobs	Registers broken device intake (Phone model, issue description, status: `Pending`).	Filter jobs by status tabs (Pending, In Progress, Finished) or search by phone model.	Update repair status, diagnostic notes, assign technician, and enter repair cost.	Admin can remove cancelled or duplicate repair job tickets.
3. Employees	Registers staff members with SHA-256 hashed password and active role.	Admin views active staff list and assigned job counts.	Edit employee contact info and activate/deactivate accounts.	Soft deletion by toggling `is_active = 0` to preserve history.

5.3 Sri Lankan Rupees (LKR) Financial Calculations

In accordance with business requirements, all repair costs and monthly financial revenue reports are strictly calculated and displayed in **Sri Lankan Rupees (LKR)**:

```
// SQL Query calculating monthly revenue exclusively for Finished jobs in LKR
SELECT COALESCE(SUM(cost), 0) AS total_revenue
FROM repair_jobs
WHERE status = 'Finished'
  AND MONTH(COALESCE(finished_at, created_at)) = @selectedMonth
  AND YEAR(COALESCE(finished_at, created_at)) = @selectedYear;

// Formatted in C++/CLI UI:
lblMonthlyIncome->Text = String::Format(L"LKR {0:N2}", totalRevenue);
```

Business Rule: Only jobs with status `Finished` are counted towards monthly income. Jobs that are `Pending`, `In Progress`, or `Couldn't Repair` are strictly excluded or have a cost of `LKR 0.00`.

6. Graphical User Interface (GUI) Screens

The assignment requires a minimum of **3 screens built in Visual Studio**. FixSphere features a clean, responsive desktop interface with **6 functional views**:

Screen / View	Primary Purpose	User Controls & Implemented Features
Screen 1: Sign-In Authentication (`LoginForm`)	Secure employee sign-in	• Text boxes for username and password. • Secure SHA-256 password hashing. • Checks active employee status in MySQL. • Clean ASCII title: *FixSphere - Employee Sign In*.
Screen 2: Executive Dashboard (`MainForm` Tab 1)	Business overview	• Real-time statistic metric cards. • Total Registered Customers. • Active Pending and In-Progress repair count. • Current Month Revenue formatted in **LKR**.
Screen 3: Customer Management (`MainForm` Tab 2)	Client records	• Customer intake form (Name, Contact, NIC, Address). • Interactive search bar with instant DataGridView filtering. • Save, Update, and Clear buttons with input validation.
Screen 4: Repair Jobs Intake & Workflow (`MainForm` Tab 3)	Core repair operations	• Intake fields: Phone Model, Defect Details, Cost (LKR). • Status Dropdown (`Pending`, `In Progress`, `Finished`, `Couldn't Repair`). • Repairer assignment dropdown (Admin-only access). • Role-filtered grid (Repairers only see their assigned devices).
Screen 5: Employee Administration (`MainForm` Tab 4)	Staff management (Admin only)	• Add new staff (Name, NIC, Contact, Address, Role). • Role selector (Admin, Checker, Repairer). • Password setup with SHA-256 generation.
Screen 6: Financial Reports & CSV Export (`MainForm` Tab 5)	Revenue accounting	• Month and Year selection dropdowns. • Dynamic revenue sum of all Finished jobs in LKR. • **Export to CSV** button for spreadsheet auditing.

6.1 Role-Based Access Control Matrix (RBAC)

Feature / Operation	Administrator	Checker (Front-Desk)	Repairer (Technician)
Register New Customers	**Yes**	**Yes**	No (Hidden)
Create New Repair Job Intake	**Yes**	**Yes**	No (Hidden)
Assign Repair Jobs to Technicians	**Yes**	No (Restricted)	No (Restricted)

Feature / Operation	Administrator	Checker (Front-Desk)	Repairer (Technician)
Update Job Status & Cost	**Any Job**	No (Read-Only)	**Only Assigned Jobs**
Manage Employees & View Reports	**Yes**	No (Hidden)	No (Hidden)

7. Final Submission Verification Checklist

This checklist verifies that every instruction and requirement in the university assignment brief has been satisfied:

Status	Requirement from Assignment PDF	Project Implementation Proof
✓	**Team of 5 Registered via Google Form**	All 5 members registered before September 17, 2026. Credentials detailed on Page 1.
✓	**Unique Small Business Scenario**	FixSphere: Mobile Phone Repair Center Management System. Distinct from typical retail/pharmacy projects.
✓	**2+ Classes using all 6 OOP Concepts**	Implemented custom domain classes: `Person`, `Customer`, `Employee`, demonstrating Class, Object, Encapsulation, Abstraction, Inheritance, and Polymorphism.
✓	**3+ GUI Screens in Visual Studio**	Built with C++/CLI Windows Forms: Login Form + 5 integrated views in MainForm (Dashboard, Customers, Jobs, Employees, Reports).
✓	**Real MySQL Database with Full CRUD**	MySQL 8.0 connection via `DatabaseService` supporting Create, Read, Update, and Delete across `customers`, `repair_jobs`, and `employees`.
✓	**Report Maximum 8 Pages**	This report is formatted to fit within the 8-page limit, containing all required diagrams and worked examples.
✓	**Public GitHub Repository URL Included**	Public repository: https://github.com/Ravindu2k4/Fixsphere containing full source code and SQL setup script.
✓	**5-Minute Live Demo Video on YouTube**	Demonstration recorded following the 5-minute dubbing script, uploaded as Unlisted, link provided in report header.

8. Conclusion

FixSphere delivers a practical software solution for mobile phone repair centers while demonstrating academic mastery of Object-Oriented Programming. By separating personal identities into an abstract `Person` base class and extending them into `Customer` and `Employee` child classes, the system showcases clean inheritance, encapsulation, and runtime polymorphism. Furthermore, by abstracting database logic inside `DatabaseService`, the Windows Forms user interface remains responsive, secure, and maintainable. With full role-based access control and financial reporting in Sri Lankan Rupees (LKR), the application fulfills all requirements specified for module **ICT 1306**.